

Phase 5

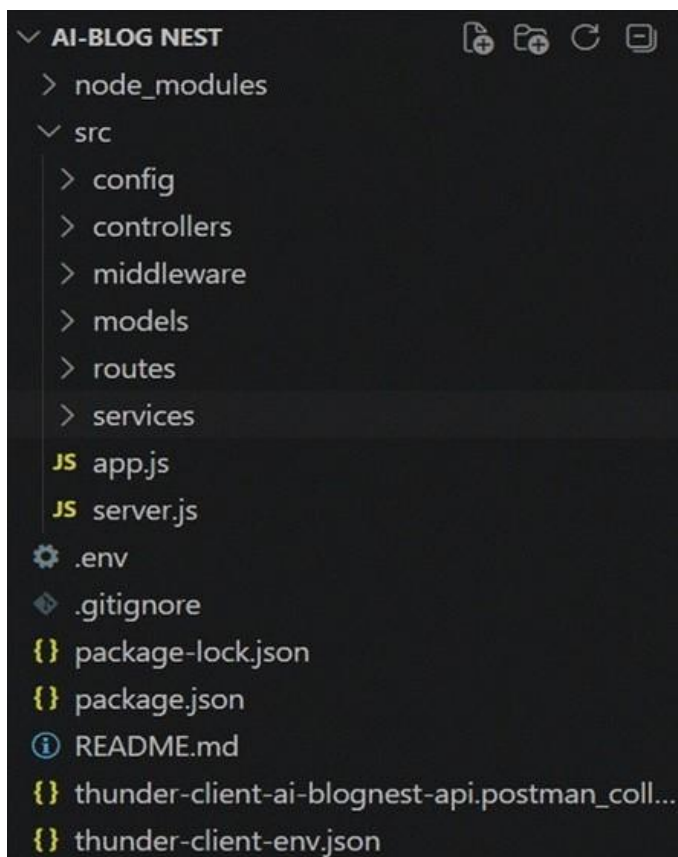
Project Development Phase

Phase 5: Project Development Phase

Covers initializing the Node.js environment, organizing the MVC folder structure in VS Code, and building the Express.js server. It details configuring the MongoDB connection using Mongoose (db.js), defining data schemas (User.js and Blog.js), setting up authentication and error middleware, and integrating Google Gemini AI services.

Creating project folder

1. Create a new folder with your <project name>
2. Inside that folder create one new folders.
3. Name another one as Server.
4. Now open that folder in VS Code.



Server setup

- Open Server folder in terminal of VScode.
 - npm init -y
- Create files:
 - server.js
- Create folders:
 - models
 - controllers
 - routes

Backend Development

Server.js

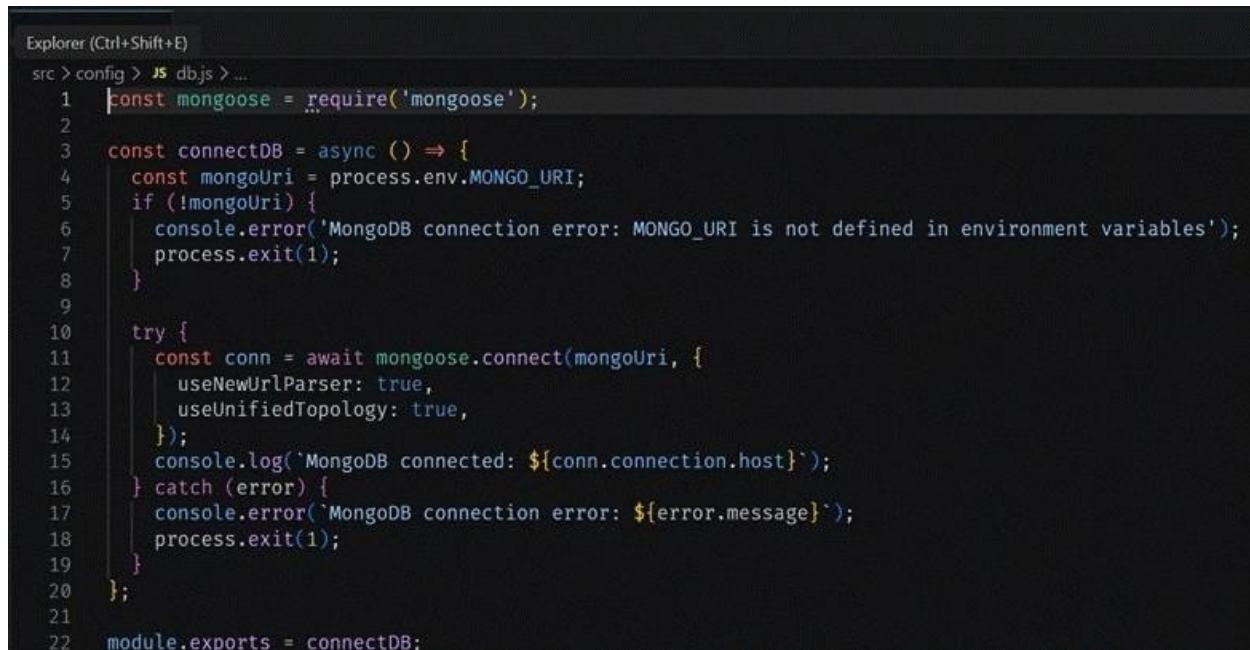
```
JS server.js  X
src > JS server.js > ...
1  const dotenv = require('dotenv');
2  const app = require('./app');
3  const connectDB = require('./config/db');
4
5  dotenv.config();
6  connectDB();
7
8  const PORT = process.env.PORT || 5000;
9  app.listen(PORT, () => {
10 |   console.log(`Server running on port ${PORT}`);
11 | });
12
```

App.js

```
JS app.js  X
src > JS app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11  app.use(cors());
12  app.use(express.json());
13  app.use(morgan('dev'));
14
15  app.use('/api/auth', authRoutes);
16  app.use('/api/blogs', blogRoutes);
17  app.use('/api/ai', aiRoutes);
18
19  app.use(errorMiddleware);
20
21  app.get('/', (req, res) => {
22 |   res.json({ message: 'AI BlogNest API is running' });
23 | });
24
25  module.exports = app;
26
```

Database Configuration

DB Connection:-



```
Explorer (Ctrl+Shift+E)
src > config > JS db.js > ...
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const mongoUri = process.env.MONGO_URI;
5    if (!mongoUri) {
6      console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7      process.exit(1);
8    }
9
10   try {
11     const conn = await mongoose.connect(mongoUri, {
12       useNewUrlParser: true,
13       useUnifiedTopology: true,
14     });
15     console.log(`MongoDB connected: ${conn.connection.host}`);
16   } catch (error) {
17     console.error(`MongoDB connection error: ${error.message}`);
18     process.exit(1);
19   }
20 };
21
22 module.exports = connectDB;
```

1. Configure MongoDB:

- Install Mongoose:
 - In your Node.js project directory, run:
 - `npm install mongoose`
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

1. Identity Resource Document Schema ([schemas/userModel.js](#))

Enforces field criteria validations on incoming database creations:

```
JS User.js  ×
src > models > JS User.js > ...
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: [true, 'Name is required'],
8        trim: true,
9      },
10     email: {
11       type: String,
12       required: [true, 'Email is required'],
13       unique: true,
14       lowercase: true,
15       trim: true,
16     },
17     password: {
18       type: String,
19       required: [true, 'Password is required'],
20       minlength: 6,
21     },
22   },
23   {
24     timestamps: true,
25   }
26 );
27
28 module.exports = mongoose.model('User', userSchema);
29
```

Project Executions:

- Install Postman or ThunderClient (VS Code extension).
- Start the backend server:
 - **npm start**

[code: Click Here TO Open Our Project Code File](#)